Experimental coding 2 — Personalisation, and the stricter question behind it

BMDS2114 Machine Learning — Group G1

Context-Aware Notification Engine (CANE): a reinforcement-learning approach to fatigue-aware push-notification pacing for user-engagement optimisation.

---

1. Purpose of this experiment

The project's central claim is that the agent learns *when* to contact each individual. That claim has a weak reading and a strong one, and they have different answers:

- **Weak (RQ2b).** Train one agent per archetype. Does each learn its own user's rhythm? Success here is personalisation *by training*.
- **Strong (RQ3).** Train a single policy across all five users, never showing it an archetype label. Does it infer who it is talking to and adapt? Success here would be personalisation *by inference*.

Both are measured against the same ground truth: the hour exhaustive search proved optimal for each person.

2. Setup: libraries, tools, and where the code lives

All studies import the shared `cane` package -- the environment, evaluation harness and agents, extracted verbatim from the four notebooks and checked for bit-for-bit parity against them. They run on **Python 3.11 / PyTorch 2.13 (CPU)**, with `numpy` and `pandas` for the recorded results and `matplotlib` for figures. Each study parallelises across cores via `--jobs`; torch is pinned to one thread per worker so the workers do not contend.

`cane/min_contact.py` implements the minimum-contact wrapper: a constraint that forces at least one contact per day, converting the unconstrained MDP into a budgeted one. It exists because coverage and timing are separable failures, and the constraint isolates the second.

In [1]:
```
import json
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
ROOT = Path.cwd()
while not (ROOT / "artifacts").is_dir() and ROOT != ROOT.parent:
    ROOT = ROOT.parent
ART = ROOT / "artifacts"

pd.set_option("display.width", 130)
pd.set_option("display.max_columns", 40)

import sys
print("reading recorded results from:", ART)
print("python", sys.version.split()[0],
      "| pandas", pd.__version__, "| numpy", np.__version__)
```

```
reading recorded results from: D:\Uni Assignments\ML\artifacts
python 3.11.15 | pandas 3.0.5 | numpy 2.4.6
```

2.1 The source that was executed

These files produced every number in this notebook. They are run from the repository root and write their output into `artifacts/`.

In [2]:
```
SOURCES = [
    "tools/personalisation_test.py",
    "tools/rq3_single_policy.py",
    "cane/min_contact.py"
]

for _p in SOURCES:
    _f = ROOT / _p
    if _f.is_file():
        _n = len(_f.read_text(encoding='utf-8').splitlines())
        print(f'{_p:38} {_n:>5} lines')
    else:
        print(f'{_p:38}   NOT FOUND')
```

```
tools/personalisation_test.py          208 lines
tools/rq3_single_policy.py             208 lines
cane/min_contact.py                    119 lines
```

3. Parameter settings and configuration

Shared across every study, so results stay comparable:

Setting	Value
Held-out evaluation episodes	seeds 900,000-900,199 (200)
Default training budget	600 episodes (~100,800 steps)
Episode length	168 steps (one week, hourly)
Actions	Hold, Engagement Nudge, Incentive Nudge

Setting	Value
Archetypes	OfficeWorker, NightOwlStudent, NightShiftWorker, NormalStudent, Housewife
Break-even click rate	0.340

Timing error is **circular**, so 23:00 and 01:00 differ by two hours rather than twenty-two. A silent cell has no chosen hour, and is recorded as `peak_hour = -1` with `hour_error = 99` rather than being dropped -- so silence stays visible in the denominators instead of quietly improving the average.

4. Ground truth: the best hour for each person

In [3]:
```
best = pd.read_csv(ART / 'best_fixed_policy.csv')
print(best.to_string(index=False))
print()
print('These target hours come from exhaustive search, not from any')
print('agent. Every timing result below is scored against them.')
```

```
      archetype  best_hour    action  reward   ctr  sends
   OfficeWorker         20    Engage    7.53 0.385    7.0
 NightOwlStudent         23 Incentive    2.18 0.547    7.0
NightShiftWorker         16    Engage    2.64 0.324    7.0
   NormalStudent         16    Engage    0.54 0.298    7.0
       Housewife         13    Engage    7.25 0.383    7.0

These target hours come from exhaustive search, not from any
agent. Every timing result below is scored against them.
```

5. Weak reading: one agent per archetype

In [4]:
```
pers = pd.read_csv(ART / 'personalisation.csv')
cols = ['agent', 'archetype', 'peak_hour', 'target_hour',
        'hour_error', 'sends_per_episode', 'reward_mean', 'ctr']
print(pers[cols].round(3).to_string(index=False))
print()
timed = pers[pers['hour_error'] != 99]
print('cells within 1h of ideal:',
      int((timed['hour_error'] <= 1).sum()), 'of', len(pers))
print('mean timing error (contacted cells):',
      round(float(timed['hour_error'].mean()), 1), 'h')
print('distinct hours chosen, best agent:',
      int(timed.groupby('agent')['peak_hour'].nunique().max()))
```

```
agent        archetype  peak_hour  target_hour  hour_error  sends_per_episode  rewar
d_mean   ctr
  DQN        Housewife          9           13           4              12.94
23.332 0.430
 DDQN        Housewife         23           13          10               7.00
-5.680 0.200
  DQN    NormalStudent         23           16           7               7.00      -
11.628 0.122
  DQN NightShiftWorker         23           16           7               7.00
-9.609 0.148
  DQN  NightOwlStudent          0           23           1               7.37
7.215 0.558
  DQN     OfficeWorker         23           20           3               7.00
-6.801 0.184
 DDQN    NormalStudent         23           16           7               7.00      -
11.628 0.122
 DDQN     OfficeWorker         23           20           3               7.00
-6.801 0.184
 DDQN  NightOwlStudent         22           23           1              17.85
2.926 0.498
 DDQN NightShiftWorker         23           16           7               7.00
-9.609 0.148

cells within 1h of ideal: 2 of 10
mean timing error (contacted cells): 5.0 h
distinct hours chosen, best agent: 3
```

6. The tuned run, which is the result the project reports

In [5]:
```
tune = pd.read_csv(ART / 'tune_study.csv')
long = tune[tune['variant'] == 'both_long']
cols = ['archetype', 'peak_hour', 'target_hour', 'hour_error',
        'sends_per_episode', 'reward_mean', 'ctr']
print(long[cols].round(3).to_string(index=False))
print()
timed = long[long['hour_error'] != 99]
print('within 1h of ideal :',
      int((timed['hour_error'] <= 1).sum()), 'of', len(long))
print('mean timing error  :',
      round(float(timed['hour_error'].mean()), 1), 'h')
print('users contacted    :', int(timed['archetype'].nunique()),
      'of', int(long['archetype'].nunique()))
print()
print('Uniform guessing averages a six-hour error, so 1.2h is a')
print('real effect. But one user was never contacted at all, and')
print('that is counted above rather than excluded.')
```

```
       archetype  peak_hour  target_hour  hour_error  sends_per_episode  reward_mean
ctr
    OfficeWorker         19           20           1             28.120       22.430
0.334
 NightOwlStudent         23           23           0             17.085       25.204
0.586
NightShiftWorker         -1           16          99              0.000        0.000
0.000
       Housewife         10           13           3             41.055       52.547
0.366
   NormalStudent         17           16           1             13.635       -0.154
0.404

within 1h of ideal : 3 of 5
mean timing error  : 1.2 h
users contacted    : 4 of 5

Uniform guessing averages a six-hour error, so 1.2h is a
real effect. But one user was never contacted at all, and
that is counted above rather than excluded.
```

7. Strong reading (RQ3): one policy, no archetype label

In [6]:

```
rq3 = pd.read_csv(ART / 'rq3_single_policy.csv')
one = rq3[(rq3['seed'] == rq3['seed'].min())
          & (rq3['agent'] == rq3['agent'].iloc[0])]
cols = ['archetype', 'peak_hour', 'target_hour', 'hour_error',
        'reward_mean', 'ctr']
print(one[cols].round(3).to_string(index=False))
print()
per_run = (rq3.groupby(['agent', 'seed'])['peak_hour']
              .nunique().rename('distinct hours').reset_index())
print(per_run.to_string(index=False))
print()
print('distinct hours across the five users:',
      sorted(rq3['peak_hour'].unique()))
print('mean timing error:',
      round(float(rq3['hour_error'].mean()), 1), 'h')
print()
print('=> RQ3 answered NO. One global schedule: the same hour for')
print('   everyone, identical across seeds and identical between')
print('   DQN and Double DQN. Without the archetype label the')
print('   policy cannot tell the users apart, so it settles on the')
print('   single hour that is least bad on average.')
```

```
       archetype  peak_hour  target_hour  hour_error  reward_mean   ctr
    OfficeWorker         23           20           3       -6.828 0.183
 NightOwlStudent         23           23           0      -14.104 0.090
NightShiftWorker         23           16           7       -9.641 0.148
   NormalStudent         23           16           7      -12.992 0.105
       Housewife         23           13          10       -6.988 0.181

agent  seed  distinct hours
 DDQN     0               1
 DDQN     1               1
  DQN     0               1
  DQN     1               1

distinct hours across the five users: [np.int64(23)]
mean timing error: 5.4 h

=> RQ3 answered NO. One global schedule: the same hour for
   everyone, identical across seeds and identical between
   DQN and Double DQN. Without the archetype label the
   policy cannot tell the users apart, so it settles on the
   single hour that is least bad on average.
```

8. Key findings and implications

1. **Personalisation by training succeeds.** Per-archetype agents reach a 1.2-hour mean timing error against a six-hour random baseline, and choose four distinct hours across five people.

2. **Personalisation by inference fails.** A single policy with no archetype label picks one hour for all five users, with a 5.4-hour mean error -- barely better than guessing. Every seed and both algorithms produce numerically identical rows, so this is a structural result rather than variance.

3. **The gap between the two is the honest finding.** The belief features recover the archetype at 72.6% accuracy offline, so the information is present in the state; it is not reaching the policy through reward alone within this training budget. Reporting only the weak reading would overstate what was demonstrated.

4. **Coverage and timing are separable failures.** The minimum-contact constraint buys coverage of every user but does not buy good timing -- cells that send exactly the seven-per-week minimum are cells where the deadline chose the hour, not the policy.